

Ex. No.: 4
System Calls Programming

Aim:

To experiment with system calls using `fork()`, `execlp()`, and `getpid()` functions.

Algorithm:

1. **Start**

Include the required header files:

```
#include <stdio.h>
#include <stdlib.h>
```

2. **Variable Declaration**

Declare an integer variable `pid` to store the process ID.

3. **Create a Process**

- Call the `fork()` function to create a new process and store the return value in `pid`.
- The return values of `fork()` are:
 - `-1`: Forking failed (child process not created)
 - `0`: Child process
 - Positive value (`> 0`): Parent process

4. **First Print Statement**

- Print the statement: `"THIS LINE EXECUTED TWICE"`.
- This line is executed by both parent and child processes after `fork()`.

5. **Check for Process Creation Failure**

- If `pid == -1`:
 - Print: `"CHILD PROCESS NOT CREATED"`
 - Exit the program using `exit(0)`.

6. **Child Process Execution**

- If `pid == 0` (child process):
 - Print the Process ID of the child using `getpid()`.
 - Print the Parent Process ID using `getppid()`.

7. **Parent Process Execution**

- If `pid > 0` (parent process):
 - Print the Process ID of the parent using `getpid()`.
 - Print the Parent's Parent Process ID using `getppid()`.

8. **Final Print Statement**

- Print the statement: `"IT CAN BE EXECUTED TWICE"`.
- This line is executed by both parent and child processes.

9. **End**

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    int pid;

    pid = fork();

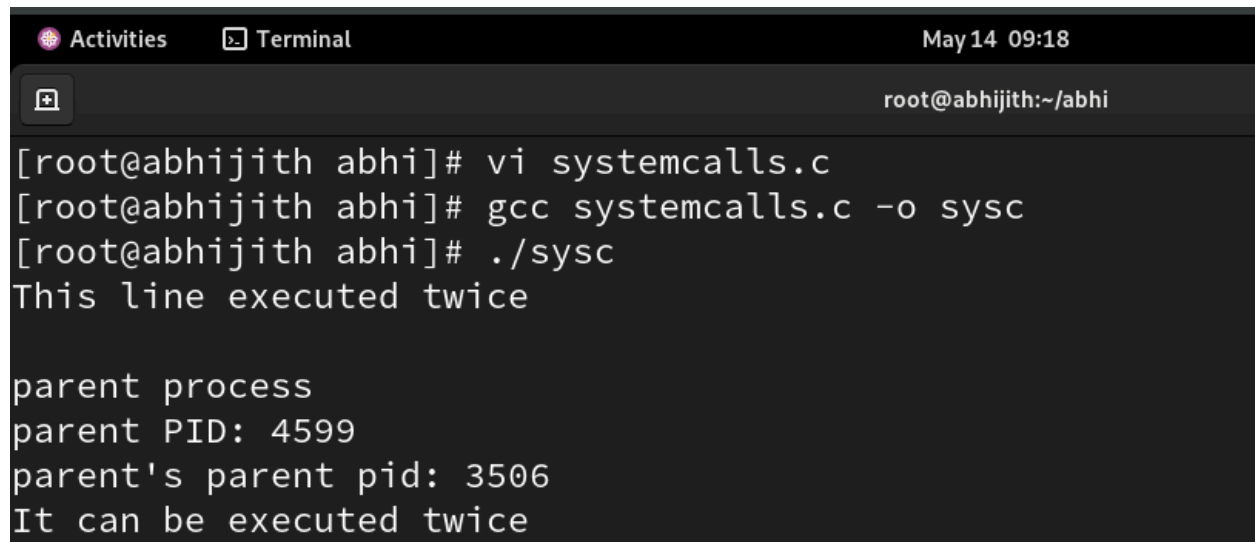
    printf("This line executed twice\n");

    if(pid == -1)
    {
        printf("CHILD PROCESS NOT CREATED\n");
        exit(0);
    }
    else if(pid == 0)
    {
        printf("\nchild process\n");
        printf("child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
    }

    if(pid == -1)
    {
        printf("CHILD PROCESS NOT CREATED\n");
        exit(0);
    }
    else if(pid == 0)
    {
        printf("\nchild process\n");
        printf("child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
    }
    else
    {
        printf("\nparent process\n");
        printf("parent PID: %d\n", getpid());
        printf("parent's parent pid: %d\n", getppid());
    }

    printf("It can be executed twice\n");

    return 0;
}
```

Output:A terminal window with a dark background. The title bar shows 'Activities', 'Terminal', and the date 'May 14 09:18'. The prompt is 'root@abhijith:~/abhi'. The user enters 'vi systemcalls.c', then 'gcc systemcalls.c -o sysc', and finally './sysc'. The output of the program is displayed below the command line.

```
[root@abhijith abhi]# vi systemcalls.c
[root@abhijith abhi]# gcc systemcalls.c -o sysc
[root@abhijith abhi]# ./sysc
This line executed twice

parent process
parent PID: 4599
parent's parent pid: 3506
It can be executed twice
```

Result:

Thus the C program to experiment with system calls using `fork()`, `execlp()`, and `getpid()` functions has been successfully executed.